PAL API Guide

This guide combines a first working model and practical modeling workflows with the detailed behavioral contracts of PAL's public API. Start with the worked sections, then use the API sections below to check semantics, constraints, and examples. For current public signatures and method descriptions, use the API Reference. For quick syntax, use the Cheatsheet. For implementation structure and design rationale, see PAL Architecture and Design.

Build a cell model, one feature at a time

This tutorial follows **one simple model**: cells occupy sites on a two-dimensional lattice. First we place and draw them. Then we let them move, divide, and die. Finally we compare a count-based representation, add nutrient diffusion, compile the timestep, and save the state. Each stage reuses the same biological idea, so the new material is about PAL rather than new biology.

Use the examples in order. The code blocks marked **complete program** can be saved as separate Python files and run independently. Later snippets show focused modifications to the preceding program. The detailed API sections remain available for complete behavior and less common features.

1. Place cells on a lattice

An `AgentGrid` stores identifiable cells and their positions. We choose a nonstackable 30 × 30 lattice: at most one cell can occupy each site. A `Grid`, by contrast, stores one typed value per site and does not create individual cell identities.

Complete program — initial cells:

```
import PythonAutomataLibrary as pal

SIZE = 30

def Setup():
    cells = pal.NewAgentGrid((SIZE, SIZE), isStackable=False)
    for x, y in ((14, 14), (15, 14), (14, 15), (15, 15)):
        cells.NewAgentSQ(x, y)
    return cells

def main():
    cells = Setup()
    print("Initial cells:", len(cells.All()))

if __name__ == "__main__":
    main()
```

`NewAgentSQ(x, y)` places a cell at lattice coordinates `(x, y)`. An agent has an identity (its handle) as well as a position. The lattice dimensions are not the number of cells: most sites are initially empty.

2. Draw the cells immediately

Visual feedback is useful from the first model. `StartPixWindow` gives us a pixel grid and a window. Each site becomes one pixel, enlarged by `scale`. We draw black background pixels and red occupied sites.

Complete program — place and display cells:

```
import time
import PythonAutomataLibrary as pal

SIZE = 30
BLACK = pal.RGB(0, 0, 0)
RED = pal.RGB(255, 0, 0)

def Setup():
    cells = pal.NewAgentGrid((SIZE, SIZE), isStackable=False)
    for x, y in ((14, 14), (15, 14), (14, 15), (15, 15)):
        cells.NewAgentSQ(x, y)
    return cells

def Draw(cells, pix):
    pix[:] = BLACK
    for agent in cells.All():
        pix[cells.XSQ(agent), cells.YSQ(agent)] = RED

def main():
    cells = Setup()
    pix, window = pal.StartPixWindow(SIZE, SIZE, scale=12, title="Cells")
    Draw(cells, pix)
    window.Update()
    pal.AwaitWindows()
    while window.IsOpen():
        time.sleep(0.05)
    window.Close()

if __name__ == "__main__":
    main()
```

You should see four red cells near the center. If you change their initial coordinates, the drawing changes accordingly. `window.Update()` publishes the pixels you wrote; drawing into `pix` alone does not refresh the display. Later we will call `Draw` after every timestep.

3. Find neighbors and empty sites

Cells divide only into empty neighboring sites. `pal.MooreHood(2, excludeCenter=True)` describes the eight relative offsets around a 2D site. `cells.Hood(..., x, y)` converts those offsets to valid lattice coordinates. This is different from a cell's continuous position: `XSQ` and `YSQ` give its lattice site.

Add this near the constants in the preceding program:

```
MOORE = pal.MooreHood(2, excludeCenter=True)

def EmptyNeighbors(cells, agent):
    sites = []
    for x, y in cells.Hood(MOORE, cells.XSQ(agent), cells.YSQ(agent)):
        if cells.counts[x, y] == 0:
            sites.append((x, y))
    return sites
```

Try printing `EmptyNeighbors(cells, cells.All()[0])` after setup. The exact result depends on the occupied sites. At a nonperiodic boundary, `Hood` omits offsets that leave the grid.

4. Let cells move, divide, and die

We now introduce one stochastic rule per timestep. Each existing cell may die; otherwise it may divide into an empty Moore neighbor, or move into one. We use a snapshot from `All(shuffle=True)`: newborn cells wait until the next timestep to act. Occupancy changes during the current step, so later cells may see sites vacated or filled by earlier cells.

Add the following constants and function to the program from stage 2, along with `MOORE` and `EmptyNeighbors` from stage 3:

```
DEATH_PROB = 0.01
BIRTH_PROB = 0.20
MOVE_PROB = 0.10

def Step(cells):
    for agent in cells.All(shuffle=True):
        if pal.Random() < DEATH_PROB:
            cells.Dispose(agent)
            continue
        empty = EmptyNeighbors(cells, agent)
        if not empty:
            continue
        x, y = empty[pal.RandInt(len(empty))]
        if pal.Random() < BIRTH_PROB:
            cells.NewAgentSQ(x, y)
        elif pal.Random() < MOVE_PROB:
            cells.MoveSQ(agent, x, y)
```

Replace the stationary display loop with:

```
    pal.AwaitWindows()
    while window.IsOpen():
        Step(cells)
        Draw(cells, pix)
        window.Update()
        time.sleep(0.05)
    window.Close()
```

Now the cells should spread, move, and occasionally disappear. This is a **sequential** agent update, not a simultaneous population update. For longer simulations, add a frame delay or render only every few steps.

5. Compile the timestep

The model behavior does not need to change to run its repeated operations through PAL's Numba integration. A compiled function must use PAL-compatible operations and types. Rather than building a Python list of neighbor coordinates, use a reusable `IList` of linear site indices.

Replace `Step` with this compiled version and construct `empty = pal.NewIList()` once in `main`. Call `Step(cells, empty)` in the display loop.

```
@pal.njit(cache=True)
def Step(cells: pal.AgentGrid, empty: pal.IList):
    for agent in cells.All(shuffle=True):
        if pal.Random() < DEATH_PROB:
            cells.Dispose(agent)
```

```
            continue
        empty.Clear()
        for x, y in cells.Hood(MOORE, cells.XSQ(agent), cells.YSQ(agent)):
            if cells.counts[x, y] == 0:
                empty.Append(cells.ToI(x, y))
        if len(empty) == 0:
            continue
        site = empty.Random()
        if pal.Random() < BIRTH_PROB:
            cells.NewAgentSQ(site)
        elif pal.Random() < MOVE_PROB:
            cells.MoveSQ(agent, cells.ItoX(site), cells.ItoY(site))
```

The `@pal.njit(cache=True)` decorator prepares supported PAL constructs for Numba. Keep the window lifecycle in ordinary Python; compile repeated model calculations. Develop in PAL's default safe mode before benchmarking fast mode.

6. Replace individual cells with population counts

The same biological processes—movement, division, and death—can be modeled without tracking individual cell identities. A `PopGrid` stores an integer count at each site, while a `Multinomial` distributes the cells at a site among competing outcomes. This is a **count-based alternative** to the preceding `AgentGrid` simulation, not an additional population.

There is one important modeling difference: `PopGrid` permits multiple cells at a site. To preserve the earlier model's one-cell-per-site rule, we would need extra rules for resolving competing arrivals and births. The example below deliberately allows multiple occupancy so that the count-based update stays simple. It uses the same movement, birth, and death probabilities, but its updates are synchronous rather than sequential; therefore the two models need not produce identical trajectories.

Complete program — count-based cells with movement, division, death, and visualization:

```
import time
import PythonAutomataLibrary as pal

SIZE = 30
DEATH_PROB = 0.01
BIRTH_PROB = 0.20
MOVE_PROB = 0.10
MOORE = pal.MooreHood(2, excludeCenter=True)
COLORS = (pal.RGB(0, 0, 0), pal.RGB(255, 0, 0), pal.RGB(255, 255, 0))

@pal.njit(cache=True)
def Step(cells: pal.PopGrid, multi: pal.Multinomial):
    for i in cells.All():
        n = cells[i]
        if n == 0:
            continue

        deaths = multi.Binomial(n, DEATH_PROB)
        cells.Add(-deaths, i)
        survivors = n - deaths

        # Each survivor may divide; new cells remain at the parent site.
        births = multi.Binomial(survivors, BIRTH_PROB)
        cells.Add(births, i)

        # Distribute the original survivors among possible destinations.
        # Probabilities refer to the starting survivor count, not the remainder.
        multi.Setup(survivors)
```

```
        for x, y in cells.Hood(MOORE, cells.ItoX(i), cells.ItoY(i)):
            moved = multi.Sample(MOVE_PROB / 8)
            cells.Add(-moved, i)
            cells.Add(moved, x, y)

    # Commit every site's changes together: incoming cells wait until next step.
    cells.Update()

def Draw(cells, pix):
    for x in range(SIZE):
        for y in range(SIZE):
            pix[x, y] = pal.ColorScale(COLORS, min(cells[x, y] / 8.0, 1.0))

def main():
    pal.Seed(1234)
    cells = pal.NewPopGrid((SIZE, SIZE))
    for x, y in ((14, 14), (15, 14), (14, 15), (15, 15)):
        cells[x, y] = 1
    multi = pal.NewMultinomial()
    pix, window = pal.StartPixWindow(SIZE, SIZE, scale=12, title="Population counts")
    pal.AwaitWindows()
    while window.IsOpen():
        Step(cells, multi)
        Draw(cells, pix)
        window.Update()
        time.sleep(0.05)
    window.Close()

if __name__ == "__main__":
    main()
```

The sampler's `Binomial(n, p)` independently draws the number of deaths or divisions. For movement, `Setup(survivors)` starts an allocation of the surviving cells. Each `Sample(MOVE_PROB / 8)` assigns cells to one of the eight neighboring directions, and unassigned cells stay in place. At a boundary, invalid directions are omitted, so those cells also stay. The two `Add` calls queue the removal and arrival for each transfer; `Update()` applies all queued births, deaths, and moves simultaneously. Unlike the individual-agent model, arrivals do not take another turn in the same timestep.

The example intentionally uses a **per-direction** movement probability of `MOVE_PROB / 8` rather than `MOVE_PROB` for every direction; otherwise eight neighbors would imply an 80% total movement probability when `MOVE_PROB = 0.10`. It also allows division without an empty neighboring site. These choices make the population-count model simple and demonstrate what is gained—and what individual occupancy detail is lost—when replacing agents with counts.

The Multinomial reference explains its probability bookkeeping and how multiple instances can share one solver core.

7. Add a diffusing nutrient

Return to the individual-cell model. A `PDEgrid` holds a continuous field on the same lattice. We start with a uniform nutrient concentration, let it diffuse, and let occupied sites consume a small amount. This introduces a second state variable without changing what a cell is.

After constructing `cells`, add:

```
nutrient = pal.NewPDEgrid((SIZE, SIZE))
nutrient.SetTimeSpaceStep(1.0, 1.0, 1.0)
for x in range(SIZE):
```

```
    for y in range(SIZE):
        nutrient[x, y] = 1.0
```

A simple field update is:

```
def StepNutrient(cells, nutrient):
    for agent in cells.All():
        x, y = cells.XSQ(agent), cells.YSQ(agent)
        nutrient[x, y] = max(0.0, nutrient[x, y] - 0.01)
    nutrient.Diffusion(0.1)
    nutrient.Update()
```

Call `StepNutrient(cells, nutrient)` once per timestep, after the cell step. Consumption here changes the current field directly; diffusion queues transport changes that become visible at `Update()`. The chosen rates are illustrative, not a calibrated biological model. You can map nutrient values to colors in a second pixel window using `pal.ColorScale`; see Visualization for the rendering API and PDEgrid for transport and boundary conditions.

8. Reproduce and save a run

Seed PAL's random stream **before** the first stochastic operation:

```
pal.Seed(1234)
```

The same seed reproduces the same random stream when the initial state and order of random calls also match. For a checkpoint, Python's `pickle` can save PAL-backed objects together with ordinary model metadata:

```
import pickle

with open("cells.pkl", "wb") as f:
    pickle.dump({"cells": cells, "nutrient": nutrient, "step": step}, f)

with open("cells.pkl", "rb") as f:
    state = pickle.load(f)
cells = state["cells"]
nutrient = state["nutrient"]
step = state["step"]
```

Saving object state is not the same as saving every source of randomness or every external resource. To resume a stochastic trajectory exactly, also preserve the relevant random state and any other model state your experiment uses. A fixed seed is sufficient for replaying a run from its beginning under the same call sequence, not necessarily for resuming halfway through it.

Where to go next

You now have the central PAL workflow: construct state, visualize it, query neighborhoods, update cells, compile repeated work, choose a representation, evolve a field, and record experiments. The remainder of this guide gives **subsystem-specific API behavior**, including specialized neighborhood forms, numerical transport methods, random sampling, and rendering options. You do not need to force those features into this simple cell model.

Core use

```
import PythonAutomataLibrary as pal
```

PAL starts in **safe mode**. Safe mode checks public operations and provides useful errors. To select fast mode:

```
pal.FastMode()
```

Call `FastMode()` before constructing any PAL model or window object. The first such construction locks the process-wide mode. Fast mode removes safety checks from performance-sensitive paths; it is intended for models already validated in safe mode.

Use `@pal.njit` for model computation:

```
@pal.njit
def Step(grid: pal.AgentGrid):
    ...
```

`pal.njit` wraps Numba `njit` and additionally transforms PAL model code. Among other PAL-specific compilation behavior, annotated PAL arguments allow safe-mode errors in compiled model code to report the model source line.

Construction

Function	Purpose
`NewGrid(dimensions, dtype)`	Typed lattice values.
`NewAgentGrid(dimensions, numAgentProps=0, isStackable=False)`	Discrete or continuous agents with optional per-agent properties.
`NewPopGrid(dimensions, capacity=None)`	Integer population counts on a lattice.
`NewPDEgrid(dimensions)`	Continuous scalar field with diffusion/advection operations.
`NewIList()`	Reusable integer query/list container.
`NewMultinomial(other=None)`	Binomial/multinomial sampler.

Spatial grids use one to three nonzero integer dimensions. Each signed dimension must fit the int32 range (excluding its minimum value), and the product of absolute axis lengths must not exceed the int32 maximum. Fractional, nonfinite, zero, oversized, or malformed dimension inputs raise `ValueError` during construction. A **negative dimension enables wrapping on that axis** while its absolute value gives the size. `AgentGrid` alone also accepts `()` for a zero-dimensional nonspatial population:

```
grid = pal.NewAgentGrid((100, 100))    # 100 x 100, no wrapping
grid = pal.NewAgentGrid((-100, 100))   # x wraps, y does not
grid = pal.NewPDEgrid((-100, -100))    # x and y wrap
```

`numAgentProps` is the number of per-agent floating-point properties. It must be a nonnegative integer fitting AgentGrid's int32 metadata budget (which also reserves entries for spatial dimensions); fractional, nonfinite, negative, or oversized values raise `ValueError`. `isStackable` accepts booleans or `0`/`1`; other values raise `ValueError`. When false, at most one agent occupies each lattice site. `PopGrid.capacity` is an optional nonnegative **per-site** population limit, not a limit on the total across the grid.

`NewMultinomial(other)` accepts another PAL Multinomial in the **same safety mode** and copies its solver configuration, not its current sampling state. An unrelated object raises `TypeError`; mixing safe and fast samplers raises `ValueError`. Create a new sampler without `other` to use the default solver.

Shared spatial API

`Grid`, `AgentGrid`, `PopGrid`, and `PDEgrid` deliberately share spatial conventions. Geometry metadata (`xDim`, `yDim`, `zDim`, `nDims`, `wrapX`, `wrapY`, `wrapZ`) is read as attributes; geometry operations such as `ToI(...)` and `Box(...)` are methods.

Geometry

Member	Meaning
`len(grid)`	Number of lattice sites.
`xDim`, `yDim`, `zDim`	Axis sizes. Missing axes are invalid in safe mode.
`nDims`	Number of spatial dimensions.
`wrapX`, `wrapY`, `wrapZ`	Whether each axis wraps.
`ToI(x, y=-1, z=-1)`	Convert coordinates to a linear lattice index.
`ItoX(i)`, `ItoY(i)`, `ItoZ(i)`	Recover coordinates from a linear index.
`InWrapX/Y/Z(value)`	Wrap an integer lattice coordinate on that axis, or return `-1` when outside a nonwrapped axis.

The same geometry names are used in 1D, 2D, and 3D. Supply only coordinates/axes that exist for the grid. `Grid`, `PopGrid`, and `PDEgrid` use `InWrapX/Y/Z` for lattice coordinates; `AgentGrid` distinguishes lattice `InWrapSQX/Y/Z` from continuous `InWrapX/Y/Z`.

Regions and neighborhoods

```
grid.Box(x1, x2)
grid.Box(x1, x2, y1, y2)
grid.Box(x1, x2, y1, y2, z1, z2)

grid.Hood(hood, x)
grid.Hood(hood, x, y)
grid.Hood(hood, x, y, z)
```

`Box` iterates coordinates in the half-open rectangular region `[x1, x2)`, `[y1, y2)`, `[z1, z2)` for the dimensions supplied. It yields an integer in 1D, `(x, y)` in 2D, and `(x, y, z)` in 3D. Bounds must be int32 integers, and the number of bound pairs must match grid dimensionality. On wrapped axes, out-of-range coordinates are wrapped; on nonwrapped axes, they are skipped.

A box wider than a wrapped axis can therefore yield repeated sites. `Hood` maps relative offsets around a lattice position to the same coordinate forms, preserving offset order and duplicates; wrapped coordinates are mapped into the grid and nonwrapped out-of-range coordinates are omitted. Offset tuple lengths must match grid dimensionality. Inside `@pal.njit`, `unroll=True` requests compile-time neighborhood unrolling without changing the returned coordinates. `Box` and `Hood` are direct-iteration constructs in compiled code: top-level `break` or `continue` from their loop body is unsupported; use explicit `range` loops when that control flow is needed.

Built-in neighborhoods:

```
pal.MooreHood(dim, excludeCenter=False)
pal.VonNeumannHood(dim, excludeCenter=False)
pal.CircleHood(dim, rad, excludeCenter=False)
```

`MooreHood` contains offsets with each coordinate in `[-1, 1]`. `VonNeumannHood` contains the center and axis-adjacent offsets. `CircleHood` contains integer offsets whose Euclidean distance from the center is at most `rad`; `rad` may be noninteger, but must be finite and nonnegative. Set `excludeCenter=True` to omit the zero offset. All three constructors support dimensions 1–3 and reject other dimension counts with `ValueError`. Each returns a tuple of integer offset tuples; these describe relative positions, not site indices, until mapped through `grid.Hood(...)`.

Grid

```
grid = pal.NewGrid((xDim, yDim), dtype)
```

Supported dtypes are `bool`, signed/unsigned 8/16/32/64-bit integers, and 32/64-bit floats.

`Grid` supports linear indexing, coordinate indexing, and NumPy-like slices:

```
grid[i]
grid[x, y]
grid[x, y, z]
grid[:] = 0
region = grid[2:5, 3:8]
```

Slice reads return detached NumPy copies. Scalar and slice reads/writes work in Python and compiled PAL model code. In safe mode, invalid scalar indices and coordinates are rejected; fast mode assumes valid access. Safe mode validates bounds, dimensionality, and whether assigned values are representable by the Grid dtype: boolean grids accept only `0`/`1`; integer grids reject fractional, nonfinite, and out-of-range assignments; floating grids reject nonfinite or out-of-range assignments. Use a wider dtype when a calculation may exceed its current range. A detached slice copy is not a live view of the Grid.

AgentGrid

```
agents = pal.NewAgentGrid((xDim, yDim), numAgentProps=2)
```

PAL represents an agent by an integer handle. Agent properties are stored on the grid and accessed with `agents[agent, property]`.

Population and lifecycle

Operation	Meaning
`GetPop()`	Number of living agents.
`Alive(agent)`	Whether an agent handle is alive.
`NewAgentSQ(...)`	Create an agent at a lattice site.
`NewAgent(...)`	Create an agent at a continuous position.
`Dispose(agent)`	Remove an agent.
`All(shuffle=False)`	Snapshot of all living agents; optionally shuffled.

`All()` has **snapshot semantics**. The returned collection can be iterated while agents are created or disposed without changing the current iteration. `All(shuffle=True)` returns the same population in randomized order.

```
for agent in agents.All():
    if ShouldDie(agent):
        agents.Dispose(agent)
```

Position

Discrete/lattice position:

```
agents.I(agent)
agents.XSQ(agent)
agents.YSQ(agent)
agents.ZSQ(agent)
agents.MoveSQ(agent, x, y, z)
```

Continuous position:

```
agents.X(agent)
agents.Y(agent)
agents.Z(agent)
agents.Move(agent, x, y, z)
```

`NewAgentSQ` and `MoveSQ` accept either a linear lattice index or lattice coordinates in every spatial dimension. SQ placement uses the center of the selected cell: after `NewAgentSQ(x, y)`, for example, `XSQ(agent)==x` and `YSQ(agent)==y`, while `X(agent)==x+0.5` and `Y(agent)==y+0.5`. Continuous `NewAgent` and `Move` instead keep the supplied spatial coordinates, with `XSQ` / `YSQ` / `ZSQ` identifying the containing cell.

Occupancy and queries

```
agents.LastAgent(x, y, z)
agents.AgentsAt(x, y, z)
agents.counts[x, y, z]
```

`LastAgent` returns the most recently stacked agent at a site, or `-1` if the site is empty. Iterate with `for agent in agents.AgentsAt(x, y):` in Python or inside `@pal.njit`. Do not create, dispose, or move agents during direct iteration, including moves within the same lattice site. Safe mode detects these changes and raises `RuntimeError`; fast mode omits that check. In Python, `AgentsAt(i)` also accepts a linear site index, including for multidimensional grids; `AgentsAt(x, y)` and `AgentsAt(x, y, z)` use coordinates. This overload is distinct from the continuous-position `AgentsInRadius` query. `counts` exposes lattice occupancy counts and supports linear-site, coordinate, and slice indexing (`agents.counts[i]`, `agents.counts[x, y]`, or `agents.counts[:, :]`). It is a **read-only query view**, not a second mutable population grid. Slice queries return detached arrays of occupancy counts. A nonspatial (`dimensions=()`) AgentGrid has no lattice counts; querying `len(agents.counts)` raises `ValueError`.

By default an `AgentGrid` is not stackable: at most one agent may occupy a lattice site. Set `isStackable=True` when multiple agents per site are required. Safe mode rejects creation or movement into an occupied site on a nonstackable grid without changing model state.

Radius iteration

`AgentsInRadius(rad, x[, y[, z]], exclude=None)` iterates agents near a continuous point in Python or inside `@pal.njit`. The yielded tuple depends on dimensionality:

```
for agent, dx in agents1.AgentsInRadius(rad, x):
    ...
for agent, dx, dy, distSq in agents2.AgentsInRadius(rad, x, y):
    ...
for agent, dx, dy, dz, distSq in agents3.AgentsInRadius(rad, x, y, z):
    ...
```

The radius is inclusive: agents with distance exactly `rad` are included. Returned displacements point from the query center to the agent and use the shortest periodic displacement on wrapped axes; `distSq` is their squared Euclidean distance in 2D/3D. `exclude` omits a specified agent handle. As with Python-side `AgentsAt`, structural modification during iteration is unsupported, and the generation check is enabled only in safe mode.

Wrapping

Discrete coordinates:

```
InWrapSQX(value)
InWrapSQY(value)
InWrapSQZ(value)
```

Continuous coordinates:

```
InWrapX(value)
InWrapY(value)
InWrapZ(value)
```

Wrapped displacement:

```
DispWrapX(x1, x2)
DispWrapY(y1, y2)
DispWrapZ(z1, z2)
```

`InWrapSQ*` maps integer lattice coordinates into the corresponding axis; `InWrap*` does the same for continuous coordinates. On a nonwrapped axis, an out-of-range coordinate returns `-1`. `DispWrap*(x1, x2)` returns the shortest signed displacement from `x1` to `x2`, accounting for periodic boundaries.

PopGrid

```
pop = pal.NewPopGrid((xDim, yDim), capacity=None)
```

`PopGrid` stores integer population counts. Indexing and direct assignment modify the current population immediately.

```
pop[x, y]
pop[x, y] = 10
pop[:] = 0
```

Linear (`pop[:]`) and spatial (`pop[:, :]`) slice reads return detached `int64` NumPy arrays; slice assignment writes to the population. Population changes can instead be accumulated and applied together:

```
pop.Add(delta, x, y)
pop.Update()
```

`Add` changes only the pending delta. Until `Update()`, `GetPop()`, indexing, and `All()` still reflect the current population. A single `Update()` commits the queued changes together, avoiding order-dependent reads during a synchronous timestep. Direct assignments during that timestep still change current values immediately.

Operation	Meaning
`GetPop()`	Total population over all sites.
`All()`	Copy of the linear indices of currently nonzero sites.
`Reset()`	Clear current populations and pending changes.

`capacity=None` uses the int64 maximum as the per-site population cap. An explicit `capacity` must be a nonnegative int64 integer; invalid, fractional, nonfinite, or out-of-range values raise `ValueError` during construction. Capacity constrains each site independently; total population is tracked separately and must remain representable as a nonnegative int64. In safe mode, direct

writes enforce these bounds immediately. `Add()` queues signed changes without applying them to the current counts; a queued change may exceed a site's remaining capacity until it is checked at `Update()`. `Update()` validates every resulting site count and the total population before applying any pending delta; if validation fails, current populations and pending deltas are unchanged. Fast mode assumes these constraints are satisfied.

PDEgrid

```
field = pal.NewPDEgrid((xDim, yDim))
```

`PDEgrid` stores a continuous `float32` scalar field. Direct indexing/assignment changes the current field immediately. Slice reads return detached `float32` NumPy arrays; array slice assignment writes to the field, but editing a previously read slice does not. Linear (`field[:]`) and spatial (`field[:, :]`) slices are supported. `Add(value, ...)` accumulates a pending delta; `Update()` applies pending changes simultaneously. `Reset()` clears both the field and pending changes.

Space and time

```
field.SetTimeSpaceStep(dt, dx, dy=1.0, dz=1.0)
field.Dt()
field.Dx()
field.Dy()
field.Dz()
```

Transport methods use these spacings. `Dy()` and `Dz()` apply only when those dimensions exist; safe mode rejects missing-dimension access. `dt` , `dx` , `dy` , and `dz` must be finite and positive in safe mode.

Boundary conditions for transport

A boundary condition specifies what happens at the edge of the modeled region. PAL's Cartesian diffusion and advection methods use the same boundary argument names, but the physical effect depends on the transport process:

Boundary choice	How to specify it	Meaning
No flux	Omit the boundary argument, or pass `None`	No material crosses that nonwrapped face
Fixed exterior concentration	Pass a scalar, such as `xMinBC=1.0`	Prescribe the concentration just outside that face
Periodic	Construct the grid with a negative dimension on that axis, such as `(-30, 30)`	The two opposite faces connect; do not pass explicit BC values on that axis

The names identify both the **axis** (`x` , `y` , or `z`) and the **side** (`Min` or `Max`). For example, `xMinBC` controls the left face and `yMaxBC` the top face of a 2D grid. Unspecified nonwrapped faces are closed by default.

For example, on a nonwrapped 2D nutrient field:

```
field = pal.NewPDEgrid((30, 30))
field.SetTimeSpaceStep(1.0, 1.0, 1.0)
```

```
field.Diffusion(0.1, xMinBC=1.0, xMaxBC=0.0)
field.Update()
```

This prescribes an exterior concentration of 1 on the left and 0 on the right, with no flux across the top and bottom. The transport operation accumulates changes; `Update()` makes them current. Start with a nonuniform field or run multiple steps to see the resulting spatial profile.

For **diffusion**, the exterior concentration determines the gradient and therefore the boundary flux. For **advection**, it supplies the concentration entering through an inflow face; on an outflow face the upwind calculation uses the interior concentration instead. Thus the same `xMinBC` argument has a different numerical role in the two processes.

For a spatially varying boundary concentration, pass a C-contiguous `float32` array containing one value per site along the face. Scalar values are simpler when the whole face has the same condition. The boundary arguments apply to Cartesian methods; radial diffusion instead uses `outerBC` for its outer radial face and symmetry at the center.

Diffusion

```
field.Diffusion(rateConstant, ...)
field.DiffusionMask(rateConstant, mask)
field.DiffusionField(rateConstants, ...)
field.DiffusionInterfaces(rateConstantsX, rateConstantsY=None, rateConstantsZ=None, ...)
field.DiffusionADI(rateConstant, ...)
field.DiffusionRadialCircle(rateConstant, outerBC=None)
field.DiffusionRadialSphere(rateConstant, outerBC=None)
```

`Diffusion` uses one constant diffusion rate. `DiffusionField` uses spatially varying cell-centered rates; each interior face uses the harmonic mean of its two adjacent cell rates, so a zero rate on either side blocks diffusion across that face. `DiffusionInterfaces` supplies the positive-face interface rate for each cell and axis directly; the negative face uses the neighboring cell's stored positive-face rate. `DiffusionMask` treats mask values `>= 0` as active and values `< 0` as blocked; diffusion occurs only between adjacent active cells, so the magnitude of a nonnegative mask value is irrelevant. `DiffusionADI` uses Crank-Nicolson in 1D, Peaceman-Rachford ADI in 2D, and Douglas-Gunn directional solves in 3D. `DiffusionRadialCircle` and `DiffusionRadialSphere` use radial shell finite-volume discretizations on a nonwrapped 1D grid with at least two cells. Cell `i` spans radii `i*dx` to `(i+1)*dx`, so cell 0 is the central disk/sphere and symmetry gives zero flux through its inner face. `outerBC` optionally sets the exterior concentration at the outer face.

Cartesian diffusion methods accept optional `xMinBC`, `xMaxBC`, `yMinBC`, `yMaxBC`, `zMinBC`, and `zMaxBC` fixed boundary values. A boundary may be a scalar applied uniformly across the face or a C-contiguous `float32` array with one value per site on that face; for example, an x-face array has `yDim*zDim` values in 3D. On a nonwrapped face, `None` means no flux through that face; wrapped axes are periodic and do not accept explicit boundary conditions. Field/interface arrays must match the grid geometry and dimensionality. Safe mode validates boundary size/type/finite values, explicit-scheme stability, and other inputs before changing pending state.

Advection

```
field.Advection(vx, vy=0.0, vz=0.0, ...)
field.AdvectionField(xVels, yVels=None, zVels=None, ...)
field.AdvectionInterfaces(xVels, yVels=None, zVels=None, ...)
```

Advection uses a conservative first-order upwind discretization. `Advection` uses constant velocity components. `AdvectionField` uses cell-centered velocity fields and averages adjacent values at interior faces. `AdvectionInterfaces` supplies positive-face velocities directly; the negative face uses the neighboring cell's stored positive-face velocity. Supply velocity components only for dimensions that exist. The same scalar-or-face-array Cartesian boundary arguments used by diffusion are available: `None` closes a nonwrapped face to advective flux, while supplied values provide the exterior concentration used for inflow. Safe mode checks the CFL condition before accumulating changes.

IList

```
items = pal.NewIList()
```

`IList` is PAL's reusable integer query/list container. Stored values are nonnegative int32 integers in the supported contract.

Operation	Meaning
`Append(i)`	Append a nonnegative int32 value; returns the IList for chaining.
`Clear()`	Remove all entries; returns the IList for chaining.
`Random()`	Return a random entry.
`Shuffle()`	Shuffle entries in place; returns the IList for chaining.
`All()`	Return a detached copy.
`Iter()`	Return a no-copy/live iterable view.
`len(items)`, `items[i]`	Length/index access.

Use `Iter()` for one-pass iteration when a copy is unnecessary. Use `All()` when the returned values must remain independent of subsequent IList changes.

Random numbers and Multinomial

```
pal.Seed(seed)
pal.Random()       # uniform float
pal.RandInt(max)  # integer in [0, max)
```

`Seed` accepts integral seeds in the uint64 domain (`0` through `2**64 - 1`); invalid or nonfinite inputs raise `ValueError`. `RandInt(max)` requires a positive integer in the int64 domain and returns an integer in `[0, max)`; zero, negative, fractional, nonfinite, or out-of-range bounds raise `ValueError`. `Random()` returns a uniform floating-point draw. PAL random functions share one seeded stream across Python and compiled PAL calls, so reseeding reproduces the same call sequence across that boundary. The stream is call-order dependent: adding or removing a random draw changes later results. This does not seed NumPy's separate RNG.

For binomial/multinomial sampling:

```
multi = pal.NewMultinomial()
x = multi.Binomial(n, p)

multi.Setup(n)
a = multi.Sample(pA)
b = multi.Sample(pB)
# remaining count/probability mass stays in the sampler
```

`Setup(n)` initializes a multinomial sequence with `n` trials and probability mass 1 and is chainable. Each `Sample(p)` takes that category's probability in the original multinomial distribution, draws from the remaining trials using the corresponding conditional probability, then removes the sampled count and `p` from the remaining state. Thus successive calls such as `Sample(0.2)` and `Sample(0.3)` represent categories with probabilities 0.2 and 0.3, not 0.2 followed by 0.3 of the remainder; any unrequested probability mass remains unsampled. In safe mode cumulative requested probability cannot exceed 1. `Binomial(n, p)` performs an independent binomial draw and does not consume the current multinomial sequence. `NewMultinomial(other)` copies solver configuration but starts with fresh sampling state.

Reusing the binomial solver core

If several independent multinomial allocations are needed, construct one sampler and pass it into another constructor:

```
movement = pal.NewMultinomial()
division = pal.NewMultinomial(movement)

movement.Setup(100)
moved = movement.Sample(0.2)

division.Setup(50)
dividing = division.Sample(0.1)
```

`NewMultinomial(movement)` **shares the underlying reusable binomial solver core**, avoiding a separate solver setup, but the two `Multinomial` objects keep **independent remaining-count and remaining-probability state**. The second instance does not inherit the first instance's partially sampled `Setup` sequence. Call `Setup` separately for each allocation. Sharing the solver does not create separate random streams: both samplers use PAL's common RNG, so results still depend on call order.

Visualization

PAL provides a pixel renderer for lattice models and an OpenGL renderer for 2D/3D geometry. Rendering runs separately from model computation.

PixWindow

```
pix, window = pal.StartPixWindow(xDim, yDim, scale=1, title="PAL", headless=False)
```

`pix` is a write-oriented pixel buffer, not a general-purpose numeric `Grid`: it supports assignment but does not expose a public pixel-read API. Assign integer RGB colors by coordinate or linear index:

```
pix[x, y] = 0xFF0000
pix[i] = 0x00FF00
```

It also provides `xDim`, `yDim`, `ToI(x, y)`, `ItoX(i)`, `ItoY(i)`, and `len(pix)`.

The window lifecycle is:

```
window.Update()
window.Save(path, block=False)
window.StartGif(path, delay=100)
window.AddGifFrame(block=False)
window.StopGif()
window.IsOpen()
window.Close()
```

`Update()` publishes the current pixel buffer to the renderer. `StartGif()` begins a GIF recording, each `AddGifFrame()` appends the currently rendered frame, and `StopGif()` finalizes the file. `block=True` on asynchronous output operations waits for that requested output to complete before returning; call `StopGif()` before relying on the GIF file as complete.

OpenGLWindow

```
draw, window = pal.StartOpenGLWindow(xDim, yDim, zDim=None,
                                     width=800, height=800,
                                     title="PAL", headless=False)
```

Draw geometry with:

```
draw.Circle(rad, color, x, y, z=0.0)
draw.Box(xLen, color, x, y, z=0.0, yLen=None, zLen=None)
draw.BoxSQ(color, x, y, z=0.0)
draw.Line(width, color, x1, y1, x2, y2, z1=0.0, z2=0.0)
```

Window controls include `Borders(width, color)`, `Camera(x, y, z, yaw=None, pitch=None)`, `Background(color)`, `Clear()`, `Update()`, `IsOpen()`, `Save(path, block=False)`, `StartGif(path, delay=100)`, `AddGifFrame(block=False, timeout=30)`, `StopGif(timeout=30)`, and `Close()`. The GIF lifecycle is the same as for `PixWindow`: start recording, append rendered frames, then stop to finalize the file.

Colors are packed integer RGB values such as `0xFF0000`. `pal.ColorScale(colors, value)` interpolates RGB channels across a sequence of colors for normalized `value`; values outside `[0, 1]` clamp to the endpoint colors.

Set `headless=True` for off-screen image/GIF generation. OpenGL headless rendering requires a supported standalone backend such as EGL.

Use `pal.AwaitWindows()` when model execution should wait for PAL windows to finish their lifecycle.

Safe and fast contracts

Safe and fast objects expose the same modeling API. PAL starts in safe mode. To opt into fast mode, call `pal.FastMode()` **before constructing any PAL model or window object**. The first such construction locks the process-wide mode selection; a later `pal.FastMode()` raises `RuntimeError`. Run safe and fast comparisons in separate processes rather than trying to switch modes mid-run. Safe mode validates public operations and is the development/default mode; fast mode assumes the model obeys those contracts and removes performance-sensitive checks. Examples of safe-mode checks include lattice bounds and dimensionality, dead-agent and occupancy errors, PopGrid overflow, and PDE field/interface array shape or dimensionality. Invalid operations in fast mode are outside the supported contract rather than an alternative error-handling API.

Use safe mode until the model is correct, then benchmark fast mode. Do not rely on safe-mode exceptions as model control flow.

Saving PAL objects

PAL native-backed model objects support standard Python pickling. Checkpoints preserve the state needed to resume the object rather than process-local pointers: AgentGrid geometry, occupancy, agents and properties; PopGrid geometry, capacity, current counts and pending deltas; PDEgrid geometry, time/space steps, current field and pending deltas; IList contents; and a Multinomial's current `Setup` / `Sample` remainder. Safe/fast identity is also restored. This means a PopGrid or PDEgrid may be pickled before `Update()` without losing its queued changes.

Choosing the right structure

Need	PAL structure
One typed value per lattice site	`Grid`
Individual agents with identity/properties	`AgentGrid`
Integer counts without individual identity	`PopGrid`
Continuous diffusing/advecting field	`PDEgrid`
Reusable integer query result	`IList`
Binomial/multinomial draws	`Multinomial`

These structures can be composed in one `@pal.njit` model step. For example, agents can query a field, queue population changes, and then update the population and field at explicitly chosen points in the timestep. The order of those commits determines which state subsequent operations read.